ENTERPRISE CLAIM MANAGEMENT SYSTEM

Generic Architecture & Data Model Reference

Applicable to Insurance, Warranty, Expense, Legal, and Employee Claim Domains

Table of Contents

1. Purpose & Design Philosophy

This document defines a domain-agnostic architecture and data model for enterprise claim management — a structure capable of supporting insurance claims, warranty claims, employee expense claims, legal claims, and any future claim type without requiring a schema rewrite or a new codebase per claim type.

A system is genuinely generic only when three concerns are kept separate and independently configurable:

- Claim data — what happened, who is involved, and how much is being claimed.
- Workflow state — where a given claim currently sits in its process.
- Workflow definition — what states and transitions are possible, and under what conditions, for a given claim type.

> *Design rule: if adding a new claim type or changing an approval step requires a code deployment, the system is not generic — it is a single workflow with a generic name.*

2. The Genericity Model

Enterprise claim systems fail in one of two directions. Overly rigid systems hardcode a single process (e.g., Employee → Manager → Finance → Payment) that cannot represent an insurance or legal claim. Overly loose systems store everything as unstructured key-value data, which becomes unqueryable and impossible to report on at scale.

The approach below picks a deliberate middle point:

- Universal, relational fields (money, dates, status, parties) are always modeled as real columns — never buried in JSON — because they must be queryable, indexable, and auditable.
- Claim-type-specific fields (e.g., vehicle VIN for auto insurance, serial number for warranty, cost center for expense) live in a validated JSON metadata field, governed by a per-claim-type JSON Schema.
- Workflow is data, not code — states, transitions, required roles, and conditions are configured per claim type in dedicated tables, evaluated by a rules/state-machine engine at runtime.

3. System Architecture — Core Modules

The module structure below is organized by capability. Each module is designed to operate against any claim type through the configuration layer described in Section 2.

3.1 Authentication & Authorization

- User login, SSO / Azure AD / SAML integration
- Role-based access control (RBAC) with attribute-based overrides (department, region, claim amount tier)
- Segregation of duties enforced at the transition level — submitter, reviewer, and approver must be distinct identities

3.2 Claim Intake

- Claim creation (First Notice of Loss / Claim Submission)
- Dynamic form rendering driven by ClaimType schema
- Draft saving, validation rules, duplicate-claim detection

- Document upload at intake

3.3 Claim Processing (Workflow Engine)

- Configurable state machine per claim type (not hardcoded)
- Conditional routing (amount thresholds, risk score, claim type)
- Manual actions: review, approve, reject, request additional information
- Escalation on SLA breach
- Reopen and dispute/appeal paths as first-class transitions

3.4 Document Management

- Attachment storage with version control
- OCR extraction with structured field mapping
- Verification status per document

3.5 Payments & Disbursement

- Partial payments, multiple disbursements, and clawbacks as separate records from the claim itself
- Idempotent integration with payment gateways / policy systems

3.6 Audit Trail

- Immutable log of all state changes, field changes to financial data, and actor identity
- Separate log for AI/automated decisions, including confidence and rationale

3.7 Notifications

- Email, SMS, Teams/Slack integration
- Templated, claim-type-aware messaging tied to workflow transitions

3.8 Reporting & Analytics

- Claim volume, approval/denial rates, SLA compliance, fraud flag rates
- Served from a reporting layer / data warehouse, not directly from OLTP tables

3.9 AI Assistant

- Document analysis and structured data extraction
- Policy / contract lookup
- Claim summarization for adjusters
- Fraud detection scoring, feeding into (not bypassing) the approval workflow

4. Core Entity Model

The schema below separates claim data, workflow configuration, workflow state, and support entities. Financial and status fields are always relational columns; type-specific attributes live in a governed JSON metadata field.

ClaimType

Defines a category of claim (insurance, warranty, expense, legal, etc.) and the schema/workflow that governs it.

Field	Description
id	Primary key
name	e.g., 'Auto Insurance', 'Product Warranty', 'Travel Expense'
description	Human-readable description
schema_definition	JSON Schema used to validate the claim's metadata field
default_workflow_id	FK to WorkflowDefinition
is_active	Whether new claims of this type can be created

Claim

The aggregate root. Fields here are universal across all claim types; anything type-specific goes into metadata.

Field	Description
id	Primary key
claim_number	Human-readable, tenant-scoped sequential identifier
tenant_id	Organization/business unit — required even for current single-tenant deployments
claim_type_id	FK to ClaimType
source_reference	Nullable reference to policy/contract/order this claim relates to
claimant_id	FK to primary claimant
status	Denormalized current state label, kept in sync with current_workflow_state_id
current_workflow_state_id	FK to WorkflowState — authoritative current position
amount_claimed	Decimal — always relational, never in JSON
amount_approved	Decimal, nullable until decisioned
currency	ISO 4217 currency code
incident_date	When the claimable event occurred
reported_date	When the claim was filed
assigned_to	FK to handler/adjuster
priority	Low / Medium / High / Critical
sla_due_at	Computed from current state's SLA rule
metadata	JSONB — claim-type-specific fields, validated against ClaimType.schema_definition
created_at / updated_at	Standard timestamps

WorkflowDefinition

A named, versioned process definition, scoped to one or more claim types.

Field	Description
id	Primary key
name	e.g., 'Standard Insurance Claim v3'
version	Integer — supports safe iteration without breaking in-flight claims
claim_type_id	FK to ClaimType
is_active	Whether new claims use this version

WorkflowState

A single node in the state machine (e.g., Intake, Under Review, Pending Approval, Closed).

Field	Description
id	Primary key
workflow_id	FK to WorkflowDefinition
name	State label
is_terminal	Whether this state ends the claim lifecycle
sla_hours	Time budget before escalation triggers

WorkflowTransition

An allowed move between two states, with the guard conditions and role required to trigger it.

Field	Description
id	Primary key
from_state_id / to_state_id	FKs to WorkflowState
required_role	Role permitted to trigger this transition
required_action_name	e.g., 'approve', 'request_info', 'escalate'
conditions	JSON — e.g., auto-route if amount_claimed < threshold

ClaimStateHistory

Append-only, immutable audit trail of every state change. Never updated in place.

Field	Description
id	Primary key
claim_id	FK to Claim
from_state_id / to_state_id	FKs to WorkflowState
actor_id	Who performed the transition
actor_type	user / system / rule_engine / ai_assistant

Field	Description
comment	Free-text rationale
payload_snapshot	Snapshot of relevant claim fields at time of transition
created_at	Timestamp

ClaimParticipant

Many-to-many link — a claim is rarely only the claimant; witnesses, approvers, and third parties all attach here.

Field	Description
id	Primary key
claim_id	FK to Claim
party_id	FK to person/organization
role	claimant / witness / adjuster / approver / third_party

ClaimDocument

Attachments with OCR and verification metadata.

Field	Description
id	Primary key
claim_id	FK to Claim
document_type	e.g., invoice, police_report, photo, receipt
file_ref	Storage pointer (S3/Blob key)
version	Version number for amended documents
uploaded_by / uploaded_at	Actor and timestamp
ocr_extracted_data	JSON — structured fields pulled from the document
verified	Boolean — whether a human confirmed OCR output

Payment

Disbursement records, kept separate from Claim to allow multiple partial payments or clawbacks.

Field	Description
id	Primary key
claim_id	FK to Claim
amount	Decimal
currency	ISO 4217 currency code
payment_method	ACH, card, check, etc.

Field	Description
status	pending / completed / failed / clawed_back
approved_by	FK to approver
external_transaction_ref	Idempotency key / gateway reference
paid_at	Timestamp

ApprovalRule

Threshold- and role-based routing, modeled as data so ops can change approval tiers without a deployment.

Field	Description
id	Primary key
claim_type_id	FK to ClaimType
min_amount / max_amount	Range this rule applies to
required_approval_tier	e.g., single-approver, dual-approver, committee
required_role	Role(s) authorized to approve at this tier

FraudSignal

Risk scoring output, feeding into — not replacing — human decisioning.

Field	Description
id	Primary key
claim_id	FK to Claim
score	Numeric risk score
model_version	Version of the scoring model used
flagged_rules	JSON — which rules/features triggered the score
reviewed_by	FK to human reviewer, nullable until reviewed

AIAuditLog

Every AI-assisted action, tied to what the AI acted on and what decision (if any) it fed.

Field	Description
id	Primary key
claim_id	FK to Claim
decision_type	e.g., document_extraction, fraud_scoring, summarization, auto_approval_recommendation
prompt / response	Input and output of the AI call

Field	Description
confidence_score	Model confidence
linked_transition_id	FK to WorkflowTransition, if the AI output triggered or gated a transition
created_at	Timestamp

5. Workflow Engine — Configured, Not Hardcoded

Rather than a single fixed sequence, each ClaimType references a WorkflowDefinition composed of WorkflowState and WorkflowTransition records. The diagram below shows a representative default — the actual states, roles, and conditions differ per claim type and are stored as data.

5.1 Representative Default States

- Draft / Intake (First Notice of Loss)
- Triage — auto-routed by claim type, amount, and risk score
- Under Investigation — documentation requested, handler assigned
- Pending Approval — routed to the approval tier defined in ApprovalRule
- Approved / Partially Approved / Denied
- Payment Processing
- Closed
- Reopened — explicit transition from Closed, with mandatory reason
- Disputed / Appealed — explicit transition from Denied

> *Reopened and Disputed states are frequently omitted in early designs and then bolted on later as workarounds. Model them from the start.*

5.2 Transition Guards

Each WorkflowTransition can carry a condition, evaluated at runtime, such as:

- amount_claimed < auto_approval_threshold → route directly to Approved
- fraud_score > risk_threshold → force route to Under Investigation regardless of amount
- required_role = 'Finance Approver' AND actor_id != submitter_id (segregation of duties)

6. Enterprise-Grade Requirements

- Multi-tenancy from day one — tenant_id on every table, even in single-tenant deployments
- Multi-currency support on all monetary fields
- Immutable audit logging sufficient for compliance and legal discovery
- Idempotent external integrations (payment gateways, policy systems) to prevent duplicate disbursement
- SLA tracking with automatic escalation on breach
- Reporting layer decoupled from transactional tables
- Configurable approval matrices and delegation of authority

- Document versioning for amended submissions

7. Anti-Patterns to Avoid

- A single Claims table with dozens of nullable columns to cover every claim type
- Status modeled as a free string with no transition table — no place to enforce which changes are valid
- Skipping the audit/history table under the assumption it can be 'added later'
- Conflating claim-type variation with tenant variation — these are separate axes and mixing them creates unmaintainable schemas
- Hardcoding a single approval chain (e.g., Manager → Finance) instead of a configurable approval matrix

8. Example: One Schema, Three Claim Types

The table below shows how the same relational schema serves three different domains purely through ClaimType configuration and metadata — no schema changes required.

Aspect	Auto Insurance	Product Warranty	Employee Expense
metadata fields	VIN, policy #, accident location	Serial #, purchase date, retailer	Cost center, project code, receipt #
Initial workflow state	FNOL Intake	Warranty Claim Intake	Draft
Approval routing	Adjuster → Risk Review → Approver	Support Agent → Auto-approve if in policy	Manager → Finance (amount-tiered)
Terminal states	Closed, Denied, Disputed	Closed, Denied	Closed, Denied

This is the practical test of genericity: adding a new claim type should mean inserting rows into ClaimType, WorkflowDefinition, WorkflowState, WorkflowTransition, and ApprovalRule — not writing new tables or redeploying code.